

Unit 1 Q&A

1. Explain the purpose of a database system.

Answer: The purpose of a database system is to provide an efficient, reliable, and convenient way to store and manage data. Before databases, data was stored in files, leading to problems like redundancy, inconsistency, and difficulty in data sharing.

A database system solves these by:

- **Data Independence:** Changes in data structure do not affect programs.
- **Reduced Redundancy:** Centralized control avoids duplicate data.
- **Improved Data Sharing:** Multiple users can access data simultaneously.
- **Integrity and Security:** Enforces rules and restricts unauthorized access.
- **Backup and Recovery:** Ensures data is restored after failure.

2. Describe the three levels of data abstraction in a DBMS.

Answer: Database systems use three levels of abstraction to simplify data interaction:

- **Physical Level:** Describes how data is stored physically (files, indexes).
- **Logical Level:** Describes what data is stored and what relationships exist between them.
- **View Level:** Provides user-specific customized views of data.

3. What is the difference between a Database schema and Database Instance/State.

Answer:

Database Schema:

- The description of a database.
- Includes descriptions of the database structure, data types, and the constraints on the database.
- The database schema changes only when the structure of the database changes.

Database State:

- The actual data stored in a database at a particular moment in time. This includes the collection of all the data in the database.
- Also called database instance (or occurrence or snapshot).
- The term instance is also applied to individual database components,
- e.g. record instance, table instance, entity instance
- The database schema changes very infrequently.
- The database state changes every time the database is updated.

4. Name the different types of database users.

Answer: The main types of database users are:

- Naive users
- Application programmers
- Sophisticated users
- Database administrator (DBA)

5. Explain the Client–Server Architecture in database systems.

Answer: Client–server architecture is a model in which the client and the server work together to process and manage data efficiently.

- **Client:** The client is the user-facing component - such as a web browser, mobile app, or desktop application. It sends requests to the server for specific data or operations and displays the results to the user.
Example: A banking app requesting account details from the server.
- **Server:** The server is the backend component that manages databases, processes client requests, executes SQL queries, and returns the results. It ensures data consistency, security, and concurrency.
Example: Database servers like MySQL, PostgreSQL, or Oracle.

6. Explain Two-Tier and Three-Tier Client–Server Architectures with examples.

Answer: In a two-tier architecture, the system is divided into two layers:

- Client (Tier 1): The user interface that sends requests and displays results.
- Server (Tier 2): The database server that processes requests and manages data.

Example:

A desktop application (client) directly communicates with a MySQL or PostgreSQL database (server).

Advantages:

- Simple and fast for small systems.
- Fewer layers → lower latency.

Disadvantages:

- Poor scalability.
- Business logic is mixed with the client or database layer.
- Difficult to maintain and secure.

In a three-tier architecture, the system is divided into three layers:

- Presentation Layer (Client): Handles the user interface (web browser, mobile app).
- Application Layer (Middle Tier): Contains business logic, validation, and processing.
- Data Layer (Server): The database that stores and retrieves data.

Example:

A web application where:

- The browser (client) sends requests,
- The application server (e.g., Node.js, Django) processes them, and
- The database server (e.g., Oracle, PostgreSQL) manages data.

Advantages:

- High scalability and maintainability.
- Better security (database not directly exposed).
- Easier to modify business logic without changing UI or database.

7. What are the types of entities in the ER model?

Answer:

1. Strong Entity: Has its own unique identifier (e.g., Student with student_id).
2. Weak Entity: Depends on a strong entity for identification (e.g., Dependent related to Employee).

8. Explain the different cardinality ratios in an ER model.

Answer:

- **One-to-one** : An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A.
- **One-to-many**: An entity in A is associated with any number (zero or more) of entities in B. An entity in B, however, can be associated with at most one entity in A.
- **Many-to-one**: An entity in A is associated with at most one entity in B. An entity in B, however, can be associated with any number (zero or more) of entities in A.
- **Many-to-many**: An entity in A is associated with any number (zero or more) of entities in B, and an entity in B is associated with any number (zero or more) of entities in A.

9. Explain recursive relationship with an example.

Answer: In a recursive relationship, the same entity type participates more than once in a relationship, but in different roles.

Example:

Entity: EMPLOYEE

Relationship: Manages

Each employee can manage other employees, forming a one-to-many relationship between the same entity.

10. What is a Weak Entity in the E-R Model?

Answer:

A Weak Entity is an entity that cannot be uniquely identified by its own attributes alone.

It depends on a Strong Entity (or Owner entity) for its identification and always has a partial key and a total participation in the identifying relationship.

Key Points:

- No primary key of its own.
- Identified through a foreign key from the strong entity.
- Denoted by a double rectangle in an E-R diagram.
- The identifying relationship is shown by a double diamond.

11. List and explain the basic operations of Relational Algebra.

Answer: Relational Algebra provides a set of operations to manipulate and retrieve data from relations (tables). The basic operations are:

1. **SELECT (σ)** – Tuple selection: Used to select rows (tuples) that satisfy a given condition.
2. **PROJECT (π)** – Column selection: Used to select specific columns (attributes) from a relation.
3. **UNION ($\cup$)** – Combines results: Combines tuples from two relations with the same schema, removing duplicates.
4. **SET DIFFERENCE ($-$)** : Finds tuples in one relation but not in another
5. **CARTESIAN PRODUCT ($\times$)**: Combines every tuple of one relation with every tuple of another
6. **RENAME (ρ)**: Renames a relation or its attributes

12. What is the use of Division operation in relational algebra?

Answer: The Division ($\div$) operation is used to find tuples in one relation that are associated with all tuples in another relation.

It is useful for queries involving “for all” conditions.

Syntax:

If $R(X, Y)$ and $S(Y)$ are relations, then $R \div S$ returns tuples of X that are related to all tuples of Y .

13. What are the types of database languages?

Answer:

- Data Definition Language (DDL) statements are used to define the database structure or schema.
- Data Manipulation Language (DML) statements are used for managing data within schema objects
- Data Control Language (DCL) statements are used to control access to the database.
- Transaction Control Language (TCL) is used to run the changes made by the DML statement.

14. What are constraints in SQL? List the constraints in SQL

Answer: Constraints enforce rules on data to maintain integrity and accuracy.

- NOT NULL: Prevents null values.
- UNIQUE: Ensures all values in a column are different.
- PRIMARY KEY: Combines NOT NULL and UNIQUE for row identification.
- FOREIGN KEY: Enforces referential integrity between tables.
- CHECK: Ensures data meets a specific condition.
- DEFAULT: Sets a default value for a column.

15. What are the different categories of data types in SQL?

Answer:

- Numeric Data Types: Used for numbers - integers and decimals.
Eg: INT, NUMERIC, FLOAT, DOUBLE
- Character/String Data Types: Used to store text or string data.
Eg: CHAR(n), VARCHAR(n), TEXT
- Date and Time Data Types: Used for temporal data.
Eg: DATE, TIME, TIMESTAMP / DATETIME
- Binary / Large Object (LOB) Data Types: Used for large binary or text data.
Eg: BLOB, CLOB

16. Explain CLOB and BLOB datatypes in SQL

Answer:

CLOB (Character Large Object):

- It is used to store large blocks of text data - for example, articles, product descriptions, comments, reports, or documents.
- Stores character/text data.
- Can hold up to several gigabytes of text, depending on the DBMS.
- Supports character set encoding (like UTF-8).
- Can be searched and manipulated with text functions.
- Commonly used in applications like blogs, document storage, or textual analytics.

BLOB (Binary Large Object)

- It is used to store binary data such as images, videos, audio, PDFs, or any file in raw binary format.
- Stores binary (non-text) data.
- Data is not human-readable.
- Used for multimedia or file storage.
- Cannot be directly searched with text functions.
- Commonly used for storing images in web apps, scanned documents, etc.

17. What is the difference between CHAR and VARCHAR data types?

Answer:

Feature	CHAR(n)	VARCHAR(n)
Type	Fixed-length string	Variable-length string
Storage	Always uses n bytes, even if the value is shorter	Uses only as many bytes as needed for the actual value
Padding	Pads extra spaces to reach the defined length	No padding is added
Performance	Slightly faster for fixed-size data	More efficient for variable-length data
Use Case	When all values have the same or similar length (e.g., country codes, gender)	When data length varies (e.g., names, email addresses)

18. What are Aggregate Functions in SQL? Explain with examples.

Answer: Aggregate functions in SQL are used to perform calculations on a set of values and return a single summarized value. They are often used with the GROUP BY clause to group rows before performing the calculation.

Function	Description	Example
COUNT()	Counts the number of rows	COUNT(*)
SUM()	Returns the total sum of a numeric column	SUM(salary)
AVG()	Returns the average value	AVG(marks)
MAX()	Returns the maximum value	MAX(price)
MIN()	Returns the minimum value	MIN(price)

19. What are Referential Actions in SQL? list the types of referential actions in SQL.

Answer: Referential actions define what happens to the rows in a child table when the corresponding row in the parent table(referenced by a foreign key) is updated or deleted.

They help maintain referential integrity between related tables.

Types of Referential Actions:

Action	Description
CASCADE	Automatically updates or deletes matching rows in the child table when a change occurs in the parent table.
SET NULL	Sets the foreign key value in the child table to NULL if the referenced row is deleted or updated.
SET DEFAULT	Sets the foreign key to its default value when the referenced row is changed or deleted.
NO ACTION	Prevents the update or delete if it would break referential integrity (default in some systems).

20. What is DDL in SQL? Give examples.

Answer: DDL (Data Definition Language) is a part of SQL used to define, modify, and delete the structure of database objects such as tables, schemas, or indexes.

Common DDL Commands:

- CREATE – creates a new table or database
- ALTER – modifies an existing table structure
- DROP – deletes a table or database
- TRUNCATE – removes all data from a table but keeps the structure

21. What is the difference between DROP and TRUNCATE commands?

Answer:

Feature	DROP	TRUNCATE
Purpose	Deletes the entire table structure and data	Deletes only the data, keeps table structure
Rollback	Cannot be rolled back in most DBs	Cannot be rolled back
Reuse table	Table no longer exists	Table can still be used
Example	DROP TABLE Student;	TRUNCATE TABLE Student;

22.Explain the ALTER command with examples.

Answer: ALTER modifies the structure of an existing table - adding, deleting, or changing columns.

Examples:

-- Add a new column

ALTER TABLE Student ADD email VARCHAR(100);

-- Modify data type

```
ALTER TABLE Student MODIFY age SMALLINT;
```

-- Drop a column

```
ALTER TABLE Student DROP COLUMN department;
```

23. What is DML in SQL? Give examples.

Answer: DML (Data Manipulation Language) consists of SQL commands used to add, modify, delete, and retrieve data stored in tables.

Main DML Commands:

- INSERT – to add new records
- UPDATE – to modify existing records
- DELETE – to remove records
- SELECT – to retrieve data

24. Describe the structure of an SQL query.

Answer: A typical SQL query has:

```
SELECT <columns>
```

```
FROM <table_name>
```

```
WHERE <condition>
```

```
GROUP BY <columns>
```

```
HAVING <group_condition>
```

```
ORDER BY <columns>;
```

25. Explain the DELETE command with an example.

Answer: The DELETE command is used to remove rows from a table.

Syntax:

```
DELETE FROM table_name
```

```
WHERE condition;
```

Example:

```
DELETE FROM Student
```

```
WHERE student_id = 1;
```

-- Removes the record of the student with ID 1.